06

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

콜백, 프로미스, async/await의 실행 순서

제1부 기반: 백엔드와 Node.js 런타임 · 2장 이벤트 루프와 비동기 I/O

비동기 코드가 실행되는 순서, 그 순서를 잘못 믿었을 때



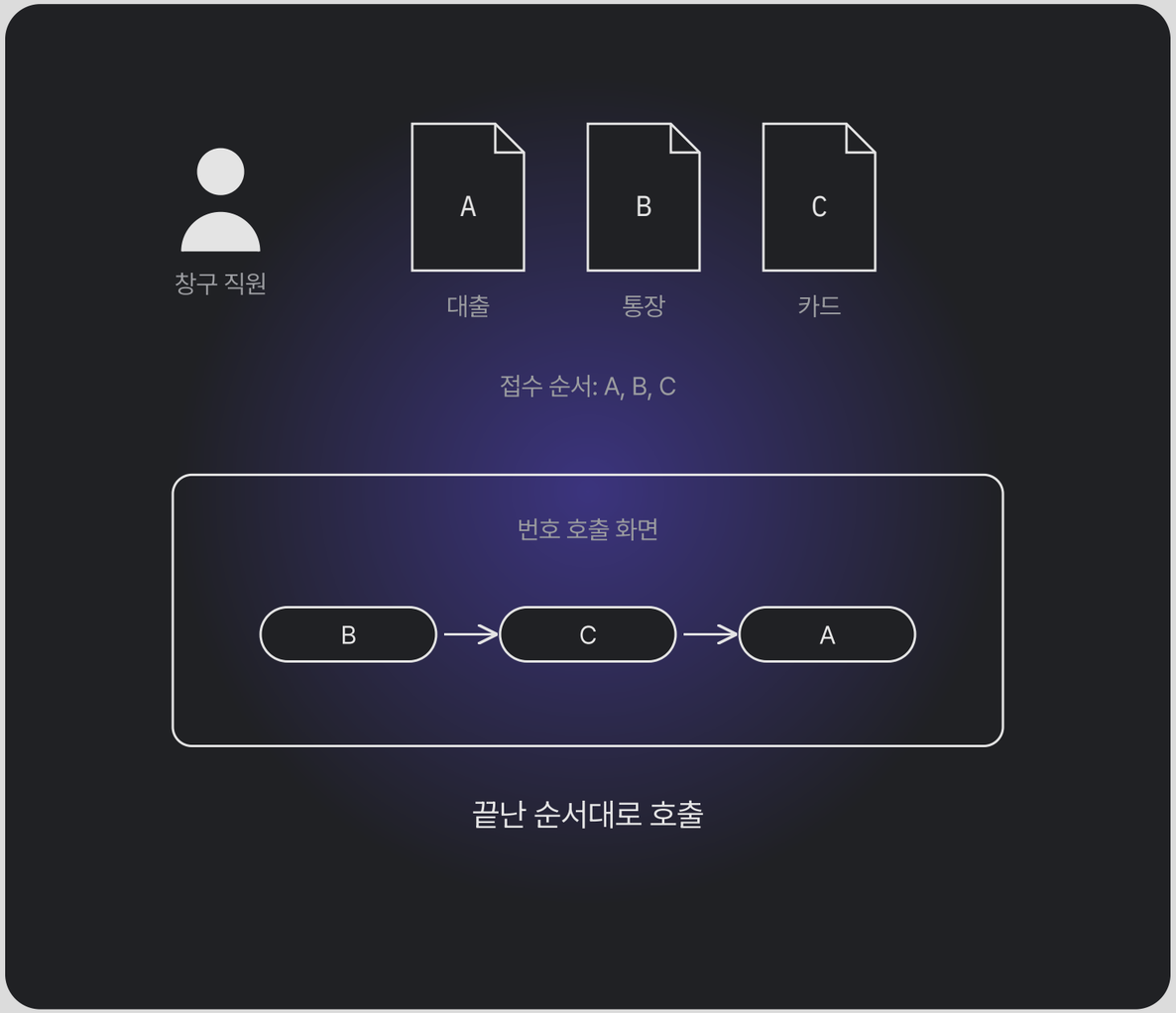
© 한상훈 NestJS 백엔드 마스터

은행 창구의 "서류 다 되면 부를게요"

→ A: 대출 서류 (오래 걸림), B: 통장 재발급 (금방), C: 카드 발급 (중간)

→ 접수 순서: A, B, C

→ 부르는 순서: B, C, A



© 한상훈 NestJS 백엔드 마스터

일곱 줄 퀴즈

order.cjs

```
1  console.log('1 동기 코드');
2  setTimeout(() => console.log('2 setTimeout 0'), 0);
3  setImmediate(() => console.log('3 setImmediate'));
4  Promise.resolve().then(() => console.log('4 Promise.then'));
5  process.nextTick(() => console.log('5 process.nextTick'));
6  queueMicrotask(() => console.log('6 queueMicrotask'));
7  console.log('7 동기 코드');
```

→ 출력 순서는?

© 한상훈 NestJS 백엔드 마스터

순서를 믿은 코드

→ 로컬에서 맞고 운영에서 틀리는 가정

저장

"저장했으니 다음 줄에서는 DB에 있음"

요청

"먼저 보낸 요청이 먼저 끝남"

타이머

"0밀리초 타이머는 바로 실행"

© 한상훈 NestJS 백엔드 마스터

콜백

콜백

콜백: 일이 끝나면 불러 달라고 넘기는 함수

넘긴 쪽

넘긴 쪽은 바로 다음 줄로

불리는 때

불리는 때: 일이 끝났을 때 (넘긴 때와 무관)

© 한상훈 NestJS 백엔드 마스터

에러 우선 콜백

```
import { readFile } from 'node:fs';

readFile('seats.json', (err, data) => {
  if (err) {
    // 실패: err에 에러, data는 없음
    return;
  }
  // 성공: err는 null, data에 결과
});
```

→ Node 콜백 API의 약속: 첫 인자는 에러

© 한상훈 NestJS 백엔드 마스터

서류 세 개

finish.cjs

```
const t0 = Date.now();
function process(name, ms, cb) { setTimeout(() => cb(null, `${name} 완료`), ms); }
process('서류 A (300ms)', 300, (err, msg) => console.log(`${Date.now() - t0}ms ${msg}`));
process('서류 B (100ms)', 100, (err, msg) => console.log(`${Date.now() - t0}ms ${msg}`));
process('서류 C (200ms)', 200, (err, msg) => console.log(`${Date.now() - t0}ms ${msg}`));
console.log('접수 끝');
```

○ ○ ○ 터미널

```
$ node finish.cjs
접수 끝
110ms 서류 B (100ms) 완료
202ms 서류 C (200ms) 완료
311ms 서류 A (300ms) 완료
```

© 한상훈 NestJS 백엔드 마스터

콜백이 겹치면

```
findUser(id, (err, user) => {
  findSeat(seatId, (err, seat) => {
    reserve(user, seat, (err, r) => {
      pay(r, (err, p) => { /* ... */ });
    });
  });
});
```

→ 순서가 필요한 일 = 콜백 안에 다음 콜백

→ 단계마다 `err` 확인

→ 콜백만 받는 API → `util.promisify`로 프로미스 함수로 변환

© 한상훈 NestJS 백엔드 마스터

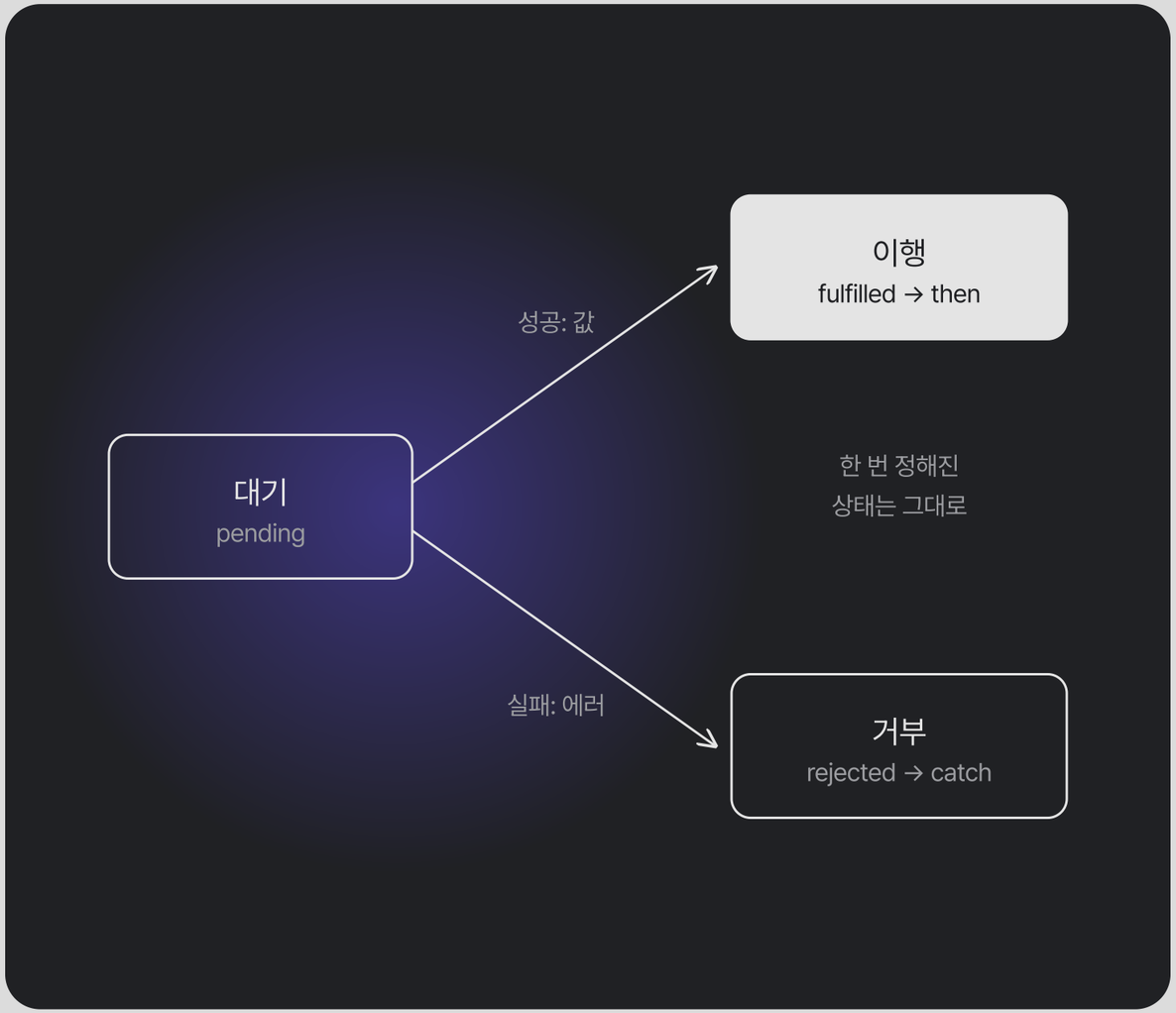
프로미스

→ 프로미스(Promise): 나중에 올 값을 담는 객체

→ 상태 셋: 대기(pending) → 이행(fulfilled) 또는 거부(rejected)

→ 한 번 정해진 상태는 그대로

→ then: 이행되면 실행할 함수, catch: 거부되면 실행할 함수



© 한상훈 NestJS 백엔드 마스터

마이크로태스크

→ `then`, `catch`에 건 함수: 마이크로태스크 큐로

→ 마이크로태스크: 지금 실행 중인 코드가 끝나면 바로 실행되는 작은 일

→ 이벤트 루프의 다음 칸으로 가기 전에 큐를 전부 비움

→ `queueMicrotask(fn)`: 마이크로태스크 큐에 직접 넣기

→ 이미 이행된 프로미스의 then도 바로 실행 안 됨: `Promise.resolve(1).then(log)` 다음 줄이 먼저

© 한상훈 NestJS 백엔드 마스터

같은 일, 세 가지 모양

```
// 콜백
readFile('seats.json', (err, data) => { /* ... */ });

// 프로미스
readFile('seats.json').then(data => { /* ... */ }).catch(err => { /* ... */ });

// async/await
try {
  const data = await readFile('seats.json');
} catch (err) { /* ... */ }
```

→ 아래 두 줄의 `readFile`: `node:fs/promises`

© 한상훈 NestJS 백엔드 마스터

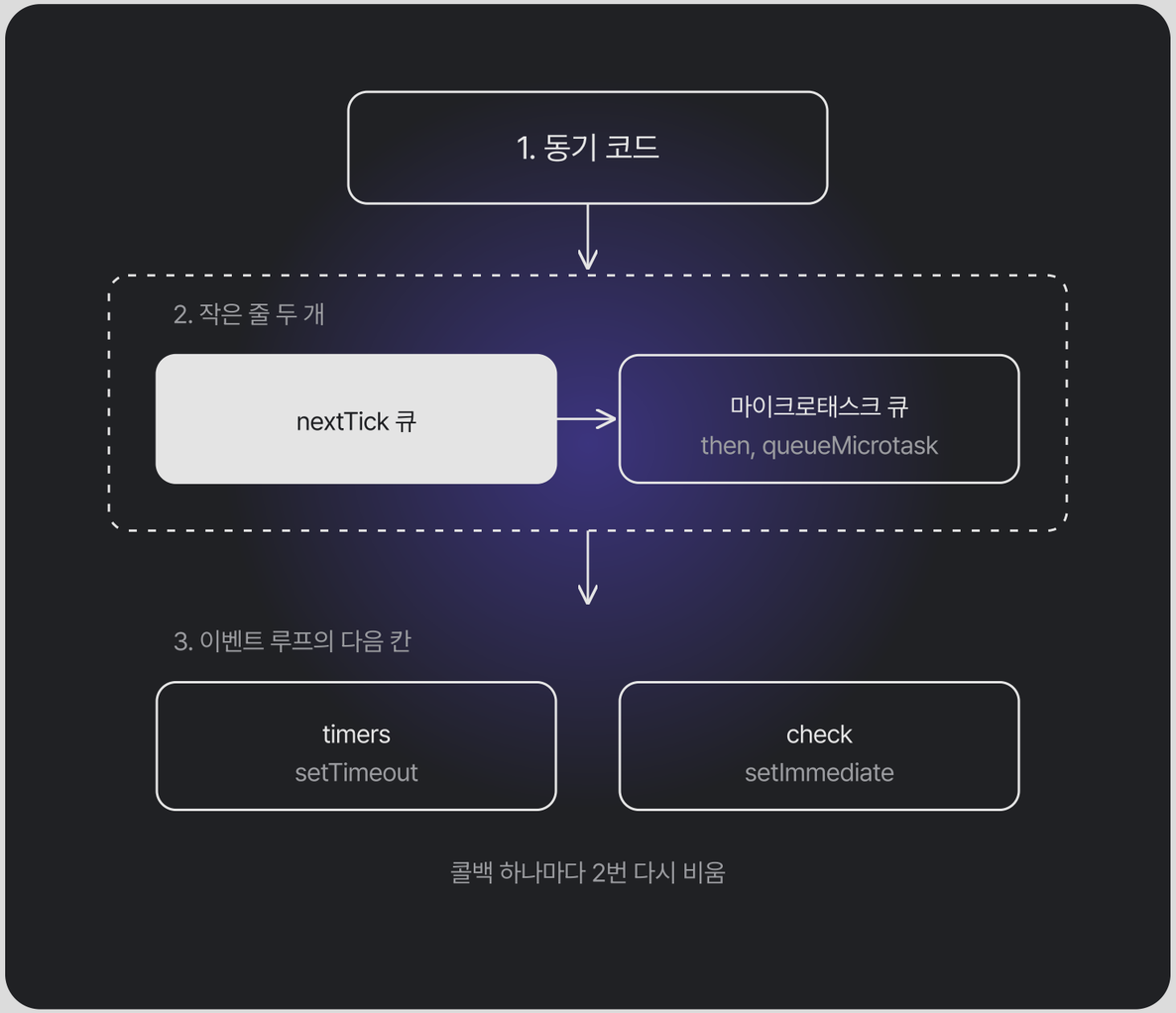
두 종류의 줄

→ 1. 지금 실행 중인 동기 코드를 끝까지

→ 2. nextTick 큐 → 마이크로태스크 큐 (`Promise.then`, `queueMicrotask`)

→ 3. 이벤트 루프의 다음 칸 (timers의 `setTimeout`, check의 `setImmediate`)

→ 콜백 하나가 끝날 때마다 2번을 다시 비움



© 한상훈 NestJS 백엔드 마스터

퀴즈의 답: CommonJS

터미널

```
$ node order.cjs
1 동기 코드
7 동기 코드
5 process.nextTick
4 Promise.then
6 queueMicrotask
2 setTimeout 0
3 setImmediate
```

→ 1, 7, 5, 4, 6, 2, 3

© 한상훈 NestJS 백엔드 마스터

setTimeout 0과 setImmediate

실행	2와 3의 순서
첫 번째	setTimeout 0 → setImmediate
두 번째	setTimeout 0 → setImmediate
세 번째	setImmediate → setTimeout 0

→ 맨 위 코드에서는 정해지지 않음

→ I/O 콜백 안에서는 항상 setImmediate 먼저

© 한상훈 NestJS 백엔드 마스터

한 줄 요약표

종류	실행되는 때
동기 코드	지금
`process.nextTick`	지금 코드가 끝난 직후 (CommonJS 기준 맨 앞)
`Promise.then`, `queueMicrotask`	그다음, 루프의 다음 칸 전
`setTimeout`, `setImmediate`	루프의 다음 칸

→ `process.nextTick`: Node 전용, 이름과 달리 다음 바퀴 전에 실행

→ `queueMicrotask`: 표준 함수

© 한상훈 NestJS 백엔드 마스터

같은 코드를 .mjs로

터미널

```
$ node order.mjs
1 동기 코드
7 동기 코드
4 Promise.then
6 queueMicrotask
5 process.nextTick
3 setImmediate
2 setTimeout 0
```

→ nextTick: 맨 앞에서 Promise 뒤로

→ 세 번 실행, 세 번 모두 같은 결과

© 한상훈 NestJS 백엔드 마스터

모듈 본문도 마이크로태스크 안

→ ES 모듈: 비동기로 불러오고 실행

→ 모듈 본문 자체가 마이크로태스크 처리 중에 실행

→ 본문이 끝나면 같은 마이크로태스크 큐를 먼저 비움 → nextTick은 그 뒤

→ 이 프로젝트: `package.json "type": "module"`

데이터 블록: Node 공식 문서 process.nextTick() vs queueMicrotask() (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

순서가 필요하면 await로

```
// 순서를 큐에 맡긴 코드
process.nextTick(() => cache.clear());
Promise.resolve().then(() => cache.load());

// 순서를 코드에 적은 코드
await cache.clear();
await cache.load();
```

→ 큐의 순서: 모듈 형식, 실행 위치, Node 버전에 따라 다름

→ `await`: 앞 줄이 끝나야 다음 줄

© 한상훈 NestJS 백엔드 마스터

async 함수가 돌아가는 때

asyncorder.mjs

```
async function reserve() {
  console.log('2 reserve 시작');
  await null;
  console.log('4 await 다음 줄');
}
console.log('1 호출 전');
reserve();
console.log('3 호출 뒤');
```

○ ○ ○ 터미널

```
$ node asyncorder.mjs
1 호출 전
2 reserve 시작
3 호출 뒤
4 await 다음 줄
```

© 한상훈 NestJS 백엔드 마스터

await 다음 줄

→ 첫 await 전까지: 부른 쪽과 같은 흐름에서 바로 실행

→ 첫 await: 함수는 프로미스를 돌려주고 부른 쪽으로 돌아감

→ await 다음 줄: 기다린 값이 오면 마이크로태스크로 이어서 실행

→ 그사이: 다른 코드, 다른 요청이 실행

→ NestJS: async 컨트롤러 메서드의 프로미스를 기다린 뒤 응답, await 빠뜨린 줄은 기다리지 않음



© 한상훈 NestJS 백엔드 마스터

비지 않는 마이크로태스크 큐

starve.mjs

```
const t0 = performance.now();
setTimeout(() => console.log(`setTimeout 0 실행: ${(performance.now() - t0).toFixed(0)}ms`), 0);
let n = 0;
function loop() {
  if (++n < 5_000_000) return Promise.resolve().then(loop);
  console.log(`마이크로태스크 ${n}개 끝: ${(performance.now() - t0).toFixed(0)}ms`);
}
loop();
```

○ ○ ○ 터미널

```
$ node starve.mjs
마이크로태스크 5000000개 끝: 867ms
setTimeout 0 실행: 924ms
```

© 한상훈 NestJS 백엔드 마스터

924ms 늦은 setTimeout 0

큐

마이크로태스크가 새 마이크로태스크를 넣으면 큐가 비지 않음

루프

큐가 빌 때까지 루프의 다음 칸으로 못 감

async

async라도 기다림 없이 이어지는 계산 = 동기 코드와 같은 멈춤

© 한상훈 NestJS 백엔드 마스터

forEach 안의 async

foreach.mjs

```
const db = [];
const save = (seat) => new Promise(r => setTimeout(() => { db.push(seat); r(); }, 50));

async function reserveSeats(seats) {
  seats.forEach(async (seat) => { await save(seat); });
  return `저장 완료, DB에 ${db.length}건`;
}
console.log(await reserveSeats(['A-3-15', 'A-3-16', 'A-3-17']));
```

© 한상훈 NestJS 백엔드 마스터

저장 완료, DB에 0건

터미널

```
$ node foreach.mjs
저장 완료, DB에 0건
100ms 뒤 DB에 3건
```

→ forEach: 콜백이 돌려준 프로미스를 기다리지 않음

→ 응답이 먼저, 저장은 나중

→ 응답 직후 조회 → "예매 내역 없음"

→ 고치는 법: `for...of` + `await`, 또는 `await Promise.all(seats.map(save))`

→ 린트 규칙 예: typescript-eslint `no-floating-promises`, `no-misused-promises`

© 한상훈 NestJS 백엔드 마스터

한 좌석이 실패하면

crash.mjs

```
async function reserveSeats(seats) {
  try {
    seats.forEach(async (seat) => { await save(seat); });  // A-3-16 저장 실패
    return '저장 완료';
  } catch (e) {
    return `실패: ${e.message}`;
  }
}
```

○ ○ ○ 터미널

```
$ node crash.mjs
저장 완료
서버는 아직 살아 있음
Error: 이미 팔린 좌석: A-3-16
Node.js v24.11.0
exit code 1
```

© 한상훈 NestJS 백엔드 마스터

처리 안 된 프로미스 거부

→ catch가 없는 프로미스의 거부 → `unhandledRejection`

→ Node 기본 동작: 프로세스 종료

→ `try/catch`: 그 함수 안에서 await한 것만 잡음

→ 서버 프로세스 종료 = 다른 손님의 요청도 끊김

© 한상훈 NestJS 백엔드 마스터

for...of와 await

```
async function reserveSeats(seats) {
  for (const seat of seats) await save(seat);
  return `저장 완료, DB에 ${db.length}건`;
}
```

○ ○ ○ 터미널

```
저장 완료, DB에 3건
```

© 한상훈 NestJS 백엔드 마스터

하나씩과 한꺼번에

방식	좌석 조회 50번 (각 20ms)
`for...of` 안에서 `await`	1,008ms
`Promise.all`	20ms

→ 서로 상관없는 조회를 줄 세우면: 50배

→ 예: 예매 화면의 공연 정보, 회차 목록, 좌석 현황, 내 쿠폰 → await 네 줄이면 네 조회 시간의 합

데이터 블록: 2026-10-26 WSL2, Node v22.22.0 실측 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

Promise.all과 Promise.allSettled

함수	하나가 실패하면	돌려주는 것
`Promise.all`	바로 거부, 나머지 결과는 버림	성공한 값 배열
`Promise.allSettled`	끝까지 기다림	상태와 값(또는 이유)의 배열

→ 실패해도 나머지는 멈추지 않음 (결과만 버려짐)

© 한상훈 NestJS 백엔드 마스터

한꺼번에 보내는 만큼

→ `Promise.all`로 쿼리 50개 → 커넥션 풀 10개 앞에 줄

→ 외부 API 50번 → 상대 서버가 동시에 받음

→ 순서가 필요한 일: `for...of` + `await`

→ 서로 상관없는 일: `Promise.all`, 동시에 보낼 개수는 따로 제한

© 한상훈 NestJS 백엔드 마스터

끝난 순서

"콜백은 끝난 순서대로 불립니다."

→ 순서가 필요하면 await로 직접 연결